MINI JUEGO “PIEDRA, PAPEL O TIJERA” CON PERSISTENCIA EN DISCO Y ESTRUCTURAS DE DATOS EN C++

Autores:

Ferrari, Ulises: Investigación, Redacción Principal y Formato APA v7

Del Pino, Lucas: Programación Backend y Pruebas de Compilación C++

Mattia, Federico: *Gestión de Repositorio Dual-Remote GitHub/GitLab*

Orué, Máximo: *Líder de Proyecto, Control de Calidad y Traza de Memoria*

Materia: Laboratorio de Programación

Institución: Escuela de Educación Secundaria Técnica N.º 1 "Eduardo Ader"

Curso: 5.° Año 3.º División (Grupos A-B)

Docente: Mansilla Muñoz York Elías y Ganduglia Yamil

Fecha de entrega: 16 de octubre de 2026

Resumen

El presente informe técnico expone la resolución analítica, conceptual y práctica correspondiente a la Actividad N.º 7 del espacio curricular Laboratorio de Programación. El trabajo se centra en la consolidación de la persistencia de datos mediante almacenamiento secundario no volátil en disco (`fstream`, `ofstream`, `ifstream`), estableciendo la diferenciación técnico-arquitectónica frente a la memoria RAM volátil. Asimismo, se aborda el modelado de información heterogénea mediante la estructura de datos `struct` en el lenguaje C++ y se documenta la estrategia de control de versiones con esquema *Dual-Remote* (sincronización simultánea en GitHub y GitLab) (EEST N.º 1 "Eduardo Ader", 2026).

Palabras clave: Persistencia en Disco, fstream, struct, ofstream, ifstream, ios::app, Dual-Remote, C++, APA v7.

1. Introducción y Objetivos

En el ámbito del desarrollo de software de bajo y medio nivel, un error conceptual recurrente entre programadores principiantes radica en la confusión entre la memoria de acceso

aleatorio (RAM) y el almacenamiento secundario en disco (EEST N.º 1 "Eduardo Ader", 2026). Mientras que la RAM actúa como un área de trabajo temporal y volátil cuyos datos se destruyen al finalizar el proceso, el almacenamiento secundario permite la conservación permanente y no volátil de la información mediante archivos físicos.

El propósito central de este trabajo es aplicar la Metodología 5R (Registrar, Reducir, Responder, Reflexionar y Revisar) para diseñar e implementar un Mini Juego de Piedra, Papel o Tijera contra una Inteligencia Artificial en consola. Cada partida disputada se registra en una estructura de datos `struct` y se almacena físicamente en un archivo de texto plano (`historial_partidas.txt`), garantizando su persistencia acumulativa a través del flujo de salida `ofstream` en modo *Append* (`ios::app`).

3. Cuestionario Técnico de Control

3.1. Memoria RAM frente a Almacenamiento Secundario

Pregunta: ¿Por qué si se cierra la ventana de la consola y se vuelve a ejecutar el programa, la Opción 2 sigue desplegando el historial de las partidas anteriores?

Respuesta: La memoria RAM es un medio de almacenamiento volátil; por ende, al finalizar la ejecución del proceso ejecutable (`.exe`), el sistema operativo libera el espacio ocupado por las variables temporales del programa. No obstante, el historial se recupera exitosamente debido a que la Opción 1 escribe físicamente los registros en el almacenamiento secundario no volátil (Disco). La Opción 2 invoca la clase `ifstream` para abrir y leer de forma secuencial el archivo `historial_partidas.txt`, restaurando los datos persistidos de sesiones previas desde el disco (EEST N.º 1 "Eduardo Ader", 2026).

3.2. Comportamiento del Modo de Apertura `ios::app`

Pregunta: ¿Qué sucedería con el archivo `historial_partidas.txt` si en la función `guardarEnDisco` se utilizara la instrucción `ofstream archivo("historial_partidas.txt");` sin especificar el indicador `ios::app`?

Respuesta: Al omitir la bandera de apertura `ios::app` (*Append*), el constructor del flujo `ofstream` asume por defecto el modo de apertura `ios::trunc` (truncamiento). Esto provoca que, al registrar una nueva partida, el archivo de texto existente se sobrescriba completamente desde el byte cero, borrando de forma irreversible todo el historial acumulado en partidas anteriores. La bandera `ios::app` es imprescindible para posicionar el puntero de escritura al final del archivo y preservar los registros existentes.

3.3. Gestión del Búfer de Entrada y `cin.ignore()`

Pregunta: ¿Por qué resulta obligatorio invocar la función `cin.ignore()` antes de capturar el nombre del jugador mediante `getline(cin, p.jugador)`?

Respuesta: La lectura de valores numéricos por medio del operador de extracción `cin >>` deja almacenado en el búfer de entrada estándar el carácter del salto de línea (`\n`, generado al presionar la tecla *Enter*). Si no se invoca `cin.ignore()` para descartar dicho carácter residual, la función `getline` lee inmediatamente ese `\n` como un fin de cadena, provocando que la variable correspondiente al nombre del jugador reciba una cadena vacía y omitiendo la interacción con el usuario.

3.4. Ventajas del Encapsulamiento con `struct`

Pregunta: ¿Qué ventajas estructurales y operativas ofrece la estructura `Partida` frente a la manipulación de variables individuales aisladas?

Respuesta: El uso de la directiva `struct` permite implementar el concepto de encapsulamiento básico, agrupando variables de tipos heterogéneos (`int` para identificadores y `string` para nombres, selecciones y resultados) bajo un mismo identificador de tipo de dato abstracto (`Partida`). Esto simplifica de manera sustancial el diseño modular, ya que permite transmitir la entidad completa como una única referencia (`const Partida& p`) hacia las funciones de guardado o procesamiento, evitando la sobrecarga y el desorden de firmas con múltiples parámetros sueltos.

3.5. Aplicación Práctica al Proyecto Integrador Anual (P.I.A.)

Pregunta: De adaptar este esquema de persistencia al Proyecto Integrador Anual (Expotécnica 2026), ¿qué tipo de eventos o datos del prototipo se almacenarían en disco?

Respuesta: La técnica de persistencia basada en registros delimitados (`.txt` o `.csv`) se adaptaría en el proyecto integrador para la creación de un sistema de *logging* y auditoría en tiempo real. Se registrarían eventos críticos del sistema como marcas de tiempo de acceso de usuarios, registros de auditoría de seguridad, modificaciones en el inventario o stock, fichajes de turnos y lecturas periódicas de sensores de telemetría de hardware, garantizando la trazabilidad e integridad de la información ante fallos o cortes imprevistos de energía eléctrica (EEST N.º 1 "Eduardo Ader", 2026).

4. Conclusiones

El desarrollo de la Actividad N.º 7 permitió contrastar experimentalmente la volatilidad del espacio de trabajo en RAM frente a la estabilidad del almacenamiento secundario en disco mediante C++. La correcta combinación de la estructura `struct Partida` para la representación en memoria y la librería `<fstream>` para el flujo de archivos delimitados por comas sentó las bases para el diseño de módulos de persistencia eficientes. Adicionalmente, el esquema de control de versiones *Dual-Remote* garantizó la trazabilidad y resiliencia del código fuente distribuido.

Referencias

EEST N.º 1 "Eduardo Ader". (2026). *Guía didáctica de Actividad 7: Mini Juego "Piedra, Papel o Tijera" con Persistencia de Datos (struct + fstream) - Laboratorio de Programación (5.° año)*. Dirección General de Cultura y Educación de la Provincia de Buenos Aires.

Ferrari, U. (2026). *Repositorio oficial del proyecto de repaso general y persistencia en C++* [Repositorio de código fuente en GitHub y GitLab]. https://github.com/Ferraribip/piedrapapelotijera.git

Repositorios de Código Fuente (Dual-Remote)

En cumplimiento con los requerimientos de resiliencia y disponibilidad del software, el proyecto fue configurado e integrado bajo una arquitectura de repositorio remoto doble (*Dual-Remote*). Las direcciones URL de acceso público a los repositorios sincronizados se detallan a continuación:

- **Repositorio Primario (GitHub - Origin):** https://github.com/Ferraribip/piedrapapelotijera.git (Ferrari, 2026)
- **Repositorio Secundario (GitLab - Gitlab):** **https://gitlab.com/Ferraribip/piedrapapelotijera.git** (Ferrari, 2026)